

Tunisian Republic
Ministry of Higher Education
and Scientific Research

University of Monastir

Private Higher School of Engineering of
Monastir - ESPRIM



Engineer In :
Computer systems and software
(SIL)

Speciality :
WEB

MEMORY

presented at

Private Higher School of Engineering of Monastir - ESPRIM

in order to obtain the

End of year project

**Specialty :
WEB**

By

**Jihed Nasr & Abir Boukhris & Firas Boudhief &
Mohamed Sandid & Mehdi bouaouina & Ahmed
Hammemi**

**Designing and implementing a Freelance Services
Management Platform**

**Mme. Samia Benismail
Mme. Hajer Maraoui**

**Supervisor
Supervisor**



Acknowledgments

We would like to express our sincere gratitude to our supervisors, Ms. Samia Benismail and Ms. Hajer Maraoui, for their invaluable guidance, support, and encouragement throughout this project. We also extend our appreciation to the **Private Higher School of Engineering of Monastir - ESPRIM** for providing us with the opportunity to enhance our skills in programming and project realization. This experience has been a significant step in our learning journey. Lastly, we would like to thank each other for the dedication and effort we put into this project, making it a valuable and enriching experience for all of us.

TABLE OF CONTENTS

LIST OF ILLUSTRATIONS	iv
GENERAL INTRODUCTION	1
1 General Project Framework	2
1.1 Introduction	3
1.2 Project Presentation :	3
1.3 Problem Statement :	3
1.4 Existing Solutions Analysis :	4
1.5 Critique of Existing Platforms :	4
1.5.1 .Upwork	4
1.5.2 .Fiverr	4
1.5.3 .Freelancer	4
1.5.4 .Malt	5
1.6 Proposed Solution and Global Objectives	5
1.6.1 Proposed Solution	5
1.6.2 Global Objectives	6
1.7 Project Management Methodology	7
1.7.1 Methodological Choice	7
1.7.2 Traditional Methods	7
1.7.3 Agile Methods	7
1.8 Definition SCRUM	9
1.9 UML Modeling Language	10
2 Planning And Architecture	12
2.1 Introduction	13
2.2 Requirements Identification	13
2.2.1 Functional requirements	13
2.2.2 Non-Functional requirements	14

TABLE OF CONTENTS

2.2.3	Technical Environment	15
2.3	General Application Architecture	17
2.3.1	System Software Specifications	17
2.4	Conclusion	18
3	Conception	19
3.1	Introduction	20
3.2	Development Of The Use-Case Model	20
3.2.1	Identification Of Actors	20
3.2.1.1	Visitor	20
3.2.1.2	Client	20
3.2.1.3	Freelancer	21
3.2.1.4	Administrator (Admin) (Platform Supervisor)	21
3.2.1.5	Super Administrator (Super Admin)	21
3.2.2	General Use-Case Diagram	22
3.2.2.1	Figure	22
3.3	A class diagram	23
3.3.0.1	Figure	23
4	Production	24
4.1	Introduction	25
4.2	DevOps Integration in the Project	30
4.3	1. Unit Testing Integration	30
4.4	2. Code Quality Analysis with SonarQube	30
4.5	Conclusion	30
	General Conclusion	31



LIST OF ILLUSTRATIONS

1.1	MVC architecture pattern	10
2.1	Vs Code logo	15
2.2	My SQL logo	15
2.3	Figma logo	16
2.4	Guithub logo	16
2.5	Creately logo	16
2.6	Angular logo	16
2.7	Spring Logo	17
2.8	MVC architecture pattern	18
3.1	Global use case diagram	22
3.2	A class diagram	23
4.1	Login / Sign in ui	25
4.2	Forget password ui	26
4.3	Mail page ui	27
4.4	Notification ui	28
4.5	Task section ui	29



GENERAL INTRODUCTION

With the rise of freelance work, more and more independent professionals are seeking efficient solutions to manage their projects, track payments, and communicate with clients. However, they often face challenges such as task management, client communication, and payment security. To address these needs, this project proposes the creation of a freelance services management platform, accessible via both a web and mobile application. This intuitive and high-performing solution will allow freelancers to organize their activities, monitor project progress, and manage their finances with ease. The platform will offer key features, including user, project, and payment management, with third-party service integrations to ensure smooth and secure transactions. Clients will also have a dedicated space to search for freelancers, place orders, and track deliverables. With a modern and user-friendly interface, this solution aims to enhance the experience of both freelancers and clients while fostering a structured and collaborative work environment.

General Project Framework

Sommaire

1.1	Introduction	3
1.2	Project Presentation :	3
1.3	Problem Statement :	3
1.4	Existing Solutions Analysis :	4
1.5	Critique of Existing Platforms :	4
1.5.1	.Upwork	4
1.5.2	.Fiverr	4
1.5.3	.Freelancer	4
1.5.4	.Malt	5
1.6	Proposed Solution and Global Objectives	5
1.6.1	Proposed Solution	5
1.6.2	Global Objectives	6
1.7	Project Management Methodology	7
1.7.1	Methodological Choice	7
1.7.2	Traditional Methods	7
1.7.3	Agile Methods	7
1.8	Definition SCRUM	9
1.9	UML Modeling Language	10

1.1 Introduction

Freelancing is growing rapidly, but managing projects, payments, and client communication remains a challenge. This project aims to develop an intuitive freelance services management platform, accessible via web and mobile, to streamline workflow and enhance productivity. The platform will offer project tracking, payment management, and secure transactions, enabling freelancers to organize their work efficiently while allowing clients to find professionals, place orders, and track deliverables. With a user-friendly interface and scalable architecture, this solution will improve collaboration and simplify freelance management.

1.2 Project Presentation :

The rise of freelancing has transformed the job market, offering professionals greater flexibility and businesses easier access to diverse talent. However, managing projects, payments, and communication between freelancers and clients remains a challenge. This project aims to create a freelance services management platform that centralizes these aspects to simplify and optimize interactions between stakeholders. Accessible via a web and mobile application, this intuitive solution will provide freelancers with the necessary tools to organize their activities, track project progress, and manage payments securely. Clients, on the other hand, will have a dedicated interface to search for freelancers, place orders, and monitor deliverables.

1.3 Problem Statement :

Despite the growing popularity of freelance work, freelancers face several challenges in managing their activities, including : Project management : Lack of a centralized tool to track assignments and organize tasks. Payment tracking : Lack of security and transparency in invoicing and receiving payments. Client communication : Scattered exchanges across different platforms, leading to information loss. Given these challenges, there is a clear need for a comprehensive and efficient solution to ensure a smooth and secure experience for both freelancers and clients.

1.4 Existing Solutions Analysis :

Currently, several freelancing platforms, such as Upwork, Fiverr, and Malt, connect freelancers with clients. However, these platforms have certain limitations : High transaction fees, reducing freelancers' earnings. Lack of flexibility in project management and personalized payments. Complex interfaces, making them difficult to use for beginners. Our project stands out by offering an optimized and accessible solution, focusing on seamless project management, secure payments, and improved freelancer-client interaction.

1.5 Critique of Existing Platforms :

1.5.1 .Upwork

.Advantages :

.Large pool of clients and freelancers. .Integrated project management tools. .Payment protection through an escrow system.

.Disadvantages : .High commission fees (up to 20).Strict profile validation process. .Intense competition, making it hard to secure projects.

1.5.2 .Fiverr

.Advantages : .Easy to create services (gigs). .High platform visibility. .Secure payment system.

.Disadvantages : .High commission fees (20).Limited flexibility in pricing. .Payment delays and long processing times.

1.5.3 .Freelancer

.Advantages :

.A wide variety of available projects. .Ability to bid on projects.

.Disadvantages :

.Auction-style bidding system leads to lower project prices. .High subscription fees and commissions. .Poor quality of some projects and clients.

1.5.4 .Malt

.Advantages : .Direct connection with companies. .Lower commission rates (5-10).Transparent pricing.

.Disadvantages :

.Limited visibility for new freelancers. .Dependent on recommendations and reviews. .Primarily available in Europe, limiting global access.

Conclusion Existing platforms provide valuable services but are often hindered by high fees, lack of flexibility, and complex user experiences. Our project aims to create a more efficient, transparent, and accessible solution for freelancers and clients alike.

1.6 Proposed Solution and Global Objectives

1.6.1 Proposed Solution

To address the challenges faced by freelancers and their clients, our project proposes the development of a freelance services management platform, accessible via both web and mobile applications. This intuitive and high-performing platform will centralize all essential features to simplify project management, communication with clients, and secure payments.

The key features will include :

User Management : Secure registration and authentication for both freelancers and clients.

Project Management : Creation, tracking, and updating of projects with a task management system.

Integrated Communication : Instant messaging and real-time notifications to streamline interactions.

Payment Tracking : Automated invoicing, a digital wallet system, and third-party service integrations for secure transactions.

Reviews and Recommendations : A rating and feedback system to ensure transparency and service quality.

Optimized User Interface : A smooth and user-friendly experience tailored to freelancers and clients.

This platform will stand out for its flexibility, lower costs compared to existing platforms, and an enhanced user experience, enabling efficient freelance service management.

1.6.2 Global Objectives

Optimize Project Management

Provide freelancers with a structured workspace to organize tasks and track project progress. Enable clients to easily manage orders and monitor deliverables. Simplify and Secure Payments

Integrate secure payment solutions (Stripe, PayPal, etc.). Automate invoicing and ensure transparent, fast transactions. Enhance Communication

Offer built-in messaging for smooth collaboration between freelancers and clients. Implement a real-time notification system. Reduce Costs and Improve Accessibility

Provide an alternative with lower commission fees compared to competitors. Ensure better visibility for new freelancers without relying solely on ratings and recommendations. Create an Intuitive and Modern User Experience

Design an ergonomic and easy-to-use interface for both freelancers and clients. Develop a mobile application for improved accessibility on the go. Promote a Collaborative and Structured Work Environment

Implement a shared workspace for freelancers and clients. Integrate analytical tools to help freelancers track their performance and earnings. With this solution, we aim to create an innovative, efficient, and accessible platform that enhances the experience for both freelancers and clients while fostering smooth and productive collaboration.

1.7 Project Management Methodology

One of the most important steps in successfully completing a project is choosing the right approach. That is why selecting the most suitable methodology is considered a crucial step to ensure proper task organization.

1.7.1 Methodological Choice

The selection of the methodology varies from one project to another based on specific criteria, including the nature and size of the project. The objectives of project management are to increase client satisfaction and reduce the complexity of the development process.

There are mainly two project management methodologies :

1.7.2 Traditional Methods

These methods are predictable and characterized by pre-organized stages in the software development lifecycle, rigid planning, and extensive documentation, such as the Waterfall model. These methods lack communication with the client and make it difficult to accommodate new requirements once the project is underway.

1.7.3 Agile Methods

Agile methods can be defined as a set of best practices in software development. Unlike traditional approaches, they are precise and user-friendly. Agile methodologies such as SCRUM and XP promote collaboration with the client, who has the opportunity to introduce modifications

throughout the project's development phases. These methods provide better visibility since the client clearly understands their needs and the priority of tasks to be completed, ensuring adaptability to change.

For our project, we have chosen an Agile approach, specifically SCRUM.

Why Use SCRUM for Our Freelance Services Management Platform ? Given the complexity and dynamic nature of a freelance services management platform, we have chosen the SCRUM methodology to guide our project development. Here's why SCRUM is the best fit for our needs :

1. Flexibility and Adaptability The freelancing industry is constantly evolving, with new user requirements emerging over time. SCRUM allows us to adapt to changing needs by implementing an iterative development process, ensuring that we can integrate feedback quickly and efficiently.

2. Incremental Delivery for Faster Results By working in sprints, we can deliver functional components of the platform incrementally. This allows early testing of key features, such as user management, project tracking, and payment processing, ensuring that we meet real-world requirements from the start.

3. Enhanced Collaboration and Transparency SCRUM promotes strong team collaboration through daily stand-up meetings and sprint reviews. This helps ensure that developers, designers, and stakeholders are always aligned, reducing miscommunication and project delays.

4. Continuous Feedback and Client Involvement Our platform targets both freelancers and clients, so user feedback is crucial. With SCRUM, we can gather user feedback frequently, refine features like the freelancer search system, order management, and secure payment integration, and make necessary improvements without disrupting the project.

5. Risk Reduction and Better Quality Control SCRUM helps identify and address potential issues early in the development cycle, minimizing risks. Through continuous testing and retrospective meetings, we ensure that the platform is stable, secure, and user-friendly at every stage.

1.8 Definition SCRUM

SCRUM is a framework used to implement the Agile methodology. It defines tasks as user stories, which describe the required features and their importance. The development process follows a series of sprints, based on short incremental iterations, aiming to deliver a functional, testable, and deployable software product. The core principle of SCRUM is to maximize productivity while ensuring client satisfaction. SCRUM is built on three key principles : transparency, inspection, and adaptation.

1. Roles in SCRUM A SCRUM team is self-organized and makes decisions on how to complete the required work to achieve the sprint goal. It consists of three main roles :

Product Owner : Responsible for the product vision, understanding all requirements, and determining the business value of each feature for the client. This role can be filled by the client or someone who represents the client's perspective.

SCRUM Master : Supports the team and the Product Owner in applying the SCRUM framework. Their responsibilities include removing obstacles and shielding the team from external disruptions.

SCRUM Team (Development Team) : Responsible for delivering the product. It is a cross-functional team that organizes itself and possesses all the necessary skills and knowledge to develop the product.

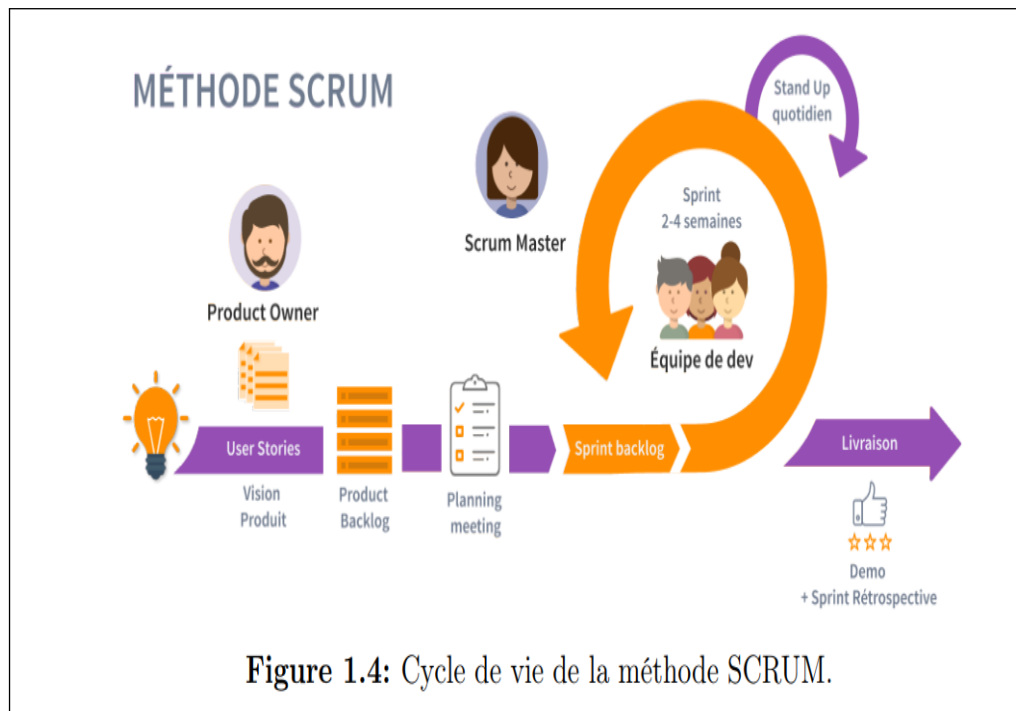


FIGURE 1.1 – MVC architecture pattern

SCRUM Tools :

Google Meet : Used for communication and meetings. **Jira** : Used for project management. **GIT/GitLab** : Used for version control and task management. **LaTeX** : Used for writing and formatting the project report.

1.9 UML Modeling Language

To model our ideas, we have chosen UML as a modeling language. Indeed, UML is a standard language used to specify, visualize, construct, and document the artifacts of software systems. In our project, we modeled our solution using the following :

1.Use Case Diagrams : They provide an overview of the system's functionalities without going into the details of the implementation. They illustrate the system's functional requirements and its interaction with external actors. A use case diagram represents different scenarios in which the system can be used.

2.Class Diagrams : They are used to describe the static structure of a system by showing the system's classes, their methods, and their attributes. Class diagrams also help to identify the relationships between these classes or objects.

Planning And Architecture

Sommaire

2.1	Introduction	13
2.2	Requirements Identification	13
2.2.1	Functional requirements	13
2.2.2	Non-Functional requirements	14
2.2.3	Technical Environment	15
2.3	General Application Architecture	17
2.3.1	System Software Specifications	17
2.4	Conclusion	18

2.1 Introduction

The second chapter, "Web Development," focuses on the design and implementation of the web platform. We will examine the functional and non-functional requirements, the system architecture, and the integration of third-party services. Additionally, we will present the overall use case diagram and the class diagram for the web component

2.2 Requirements Identification

2.2.1 Functional requirements

i. Visitor :

- **Browse the platform.**
- **Perform a search.**
- **Create an account.**

ii. Client :

- **Publish a task.**
- **Contact a freelancer.**
- **Rate a freelancer.**
- **Make payments for services.**
- **Track the progress of projects.**

iii. Freelancer :

- **Contact a client.**

- **Respond to posted tasks.**
- **Comment on a task.**
- **Subscribe to the platform.**
- **Pay for a subscription to access premium features.**

iv. Administrator :

- **Manage administrators :** Add a new administrator, Modify administrator details, Delete an administrator.
- **View platform statistics.**
- **Manage freelancer profiles :** View freelancer profiles, Delete freelancer profiles if needed.
- **Manage client profiles :** View client profiles, Delete client profiles if needed.
- **Review client needs.**
- **Share important information on the platform.**

v. Super Administrator :

- **Perform all administrator tasks with extended permissions.**
- **Oversee all platform activities.**
- **Access advanced moderation and management tools.**

2.2.2 Non-Functional requirements

The basic standards that outline the system are its non-functional requirements. maintain the smooth operation of our application. Therefore, it ought to be able to fulfill the following requirements :

- **-Performance :** Fast response times to ensure a positive user experience.

-Security :Protection of personal data and transactions through robust security protocols (SSL, payment encryption).

-Accessibility : : The platform should be accessible on both the web and via a cross-platform mobile app.

-Scalability : The platform should be able to support a large number of users simultaneously.

2.2.3 Technical Environment

In this part, we will present the different tools, development languages and frameworks that we have used for our project :

Softwares

- Visual Studio Code

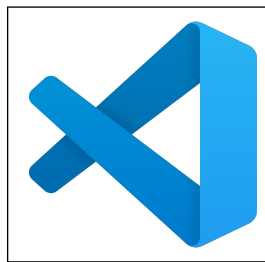


FIGURE 2.1 – Vs Code logo

- My SQL



FIGURE 2.2 – My SQL logo

- Figma : For UI/UX

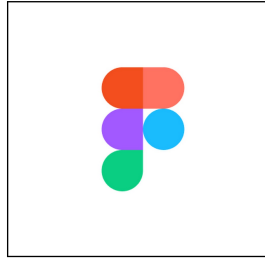


FIGURE 2.3 – Figma logo

website

- Github

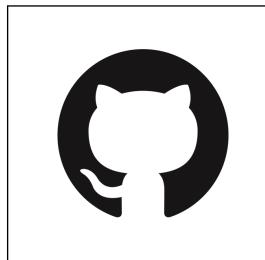


FIGURE 2.4 – Github logo

- Creately : For UML



FIGURE 2.5 – Creately logo

Front end

- Angular



FIGURE 2.6 – Angular logo

- Html,css

Back end

- Spring



FIGURE 2.7 – Spring Logo

2.3 General Application Architecture

2.3.1 System Software Specifications

MVC (Controller View Model) is a development architecture designed to divide the source code into modules. So, as you can see, there are 3 different layers :

- **Model** : is in charge of data access and manipulation. The Model understands how to work with the data contained in the application. It also understands how to respond to information requests. The Model also reflects the underlying, logical structure of data in an application, as well as the high-level classes that go with it.

- **View** : The View shows the user the data (from the Model). The information is shown by the UI and the component. It is a set of classes that represent the user interface components.

- **Controller** : This section serves as an intermediary between the model and the view : the controller requests data from the model, analyzes it, and processes it based on the parameter specified by the view in order to send the text to the display. Because of its simplicity, the MVC pattern is extremely powerful for mobile application development. It is for this reason that we have chosen to use this design.

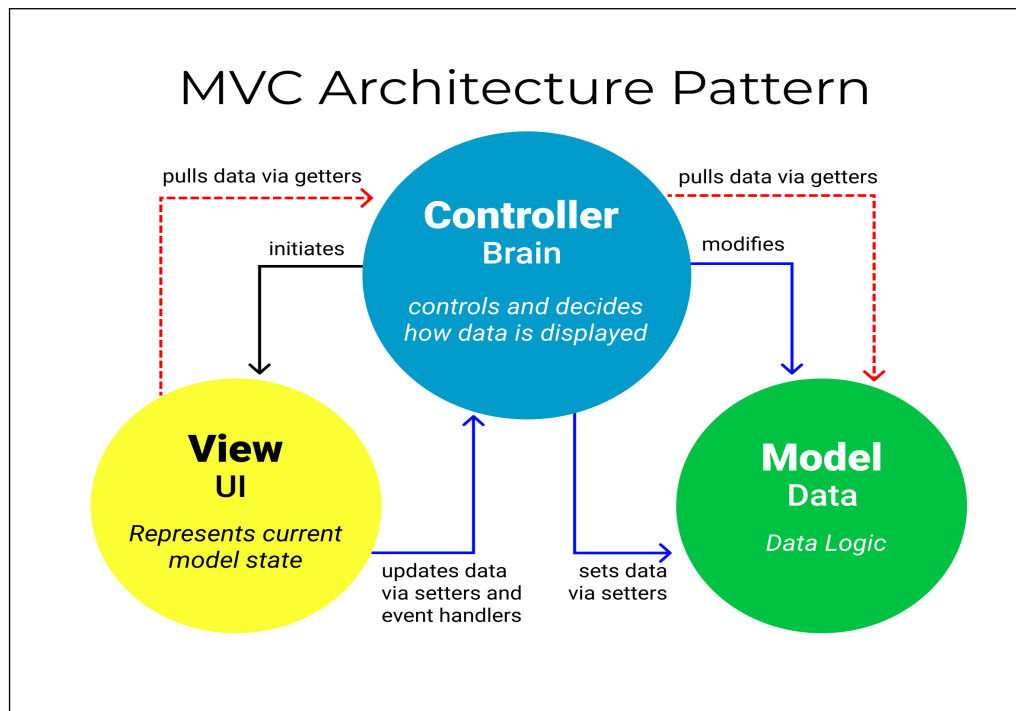


FIGURE 2.8 – MVC architecture pattern

2.4 Conclusion

In this chapter, we have presented the requirements specifications and the working tools used in our project. In the next chapter, we move on to the conception stage to better understand the interactions between the user and the targeted site.

Conception

Sommaire

3.1	Introduction	20
3.2	Development Of The Use-Case Model	20
3.2.1	Identification Of Actors	20
3.2.2	General Use-Case Diagram	22
3.3	A class diagram	23

3.1 Introduction

Throughout this chapter, we will develop general and detailed use case diagrams to better identify the interaction between the actors and the system as well as their textual description, then we will develop sequence diagrams, global class diagrams and component diagrams.

3.2 Development Of The Use-Case Model

3.2.1 Identification Of Actors

3.2.1.1 Visitor

Role : A non-registered user who can browse the platform and view general information.

Actions :

- Browse the platform.
- Perform a search.
- Create an account.

3.2.1.2 Client

Role : A registered user who posts freelance tasks and manages collaborations.

Actions :

- Publish a task.
- Contact a freelancer.
- Rate a freelancer.
- Make payments for services.
- Track the progress of projects.

3.2.1.3 Freelancer

Role : A registered user who offers freelance services and responds to client requests.

Actions :

- Contact a client.
- Respond to posted tasks.
- Comment on a task.
- Subscribe to the platform.
- Pay for a subscription to access premium features.

3.2.1.4 Administrator (Admin) (Platform Supervisor)

Role : Manages the platform, user accounts, and content moderation.

Actions :

- Manage administrators (add, modify, delete).
- View platform statistics.
- Manage freelancer profiles (view, delete).
- Manage client profiles (view, delete).
- Review client needs.
- Share important information on the platform.

3.2.1.5 Super Administrator (Super Admin)

Role : Oversees the entire platform with full control and advanced permissions.

Actions :

- Perform all administrator tasks.
- Oversee all platform activities.
- Access advanced moderation and management tools.

3.2.2 General Use-Case Diagram

Use case diagrams (UCD) are UML diagrams used to represent the functional behaviour of a software system. They allow to describe the interaction between the actor and the system. In the same context we will present the general use case diagram to give an overall view of the functional behaviour of our project. This is presented in the figure below.

3.2.2.1 Figure

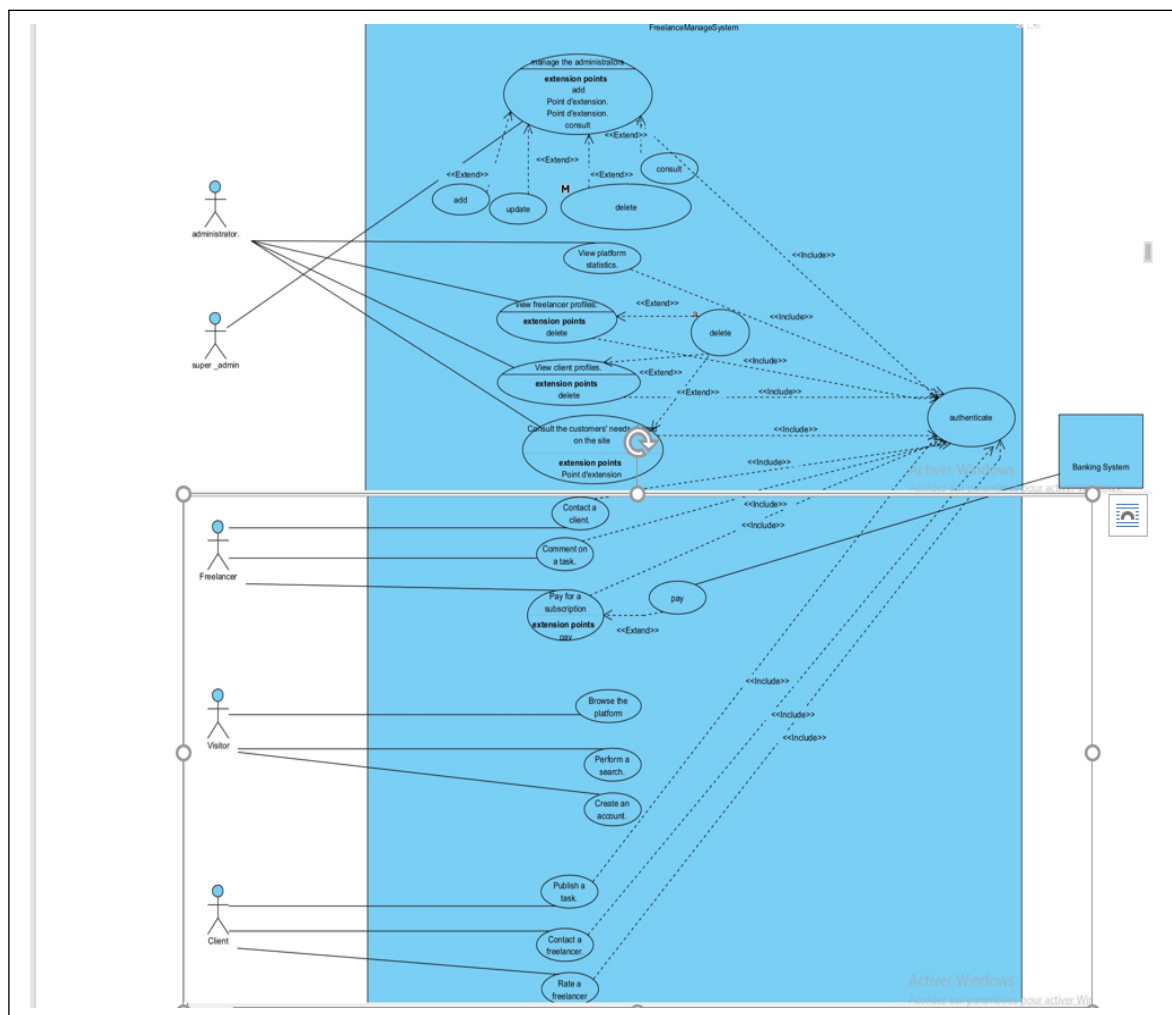


FIGURE 3.1 – Global use case diagram

3.3 A class diagram

A class diagram is a UML diagram that represents a system's static structure, showing classes, attributes, methods, and relationships between them. It is essential for object-oriented modeling in software engineering.

3.3.0.1 Figure

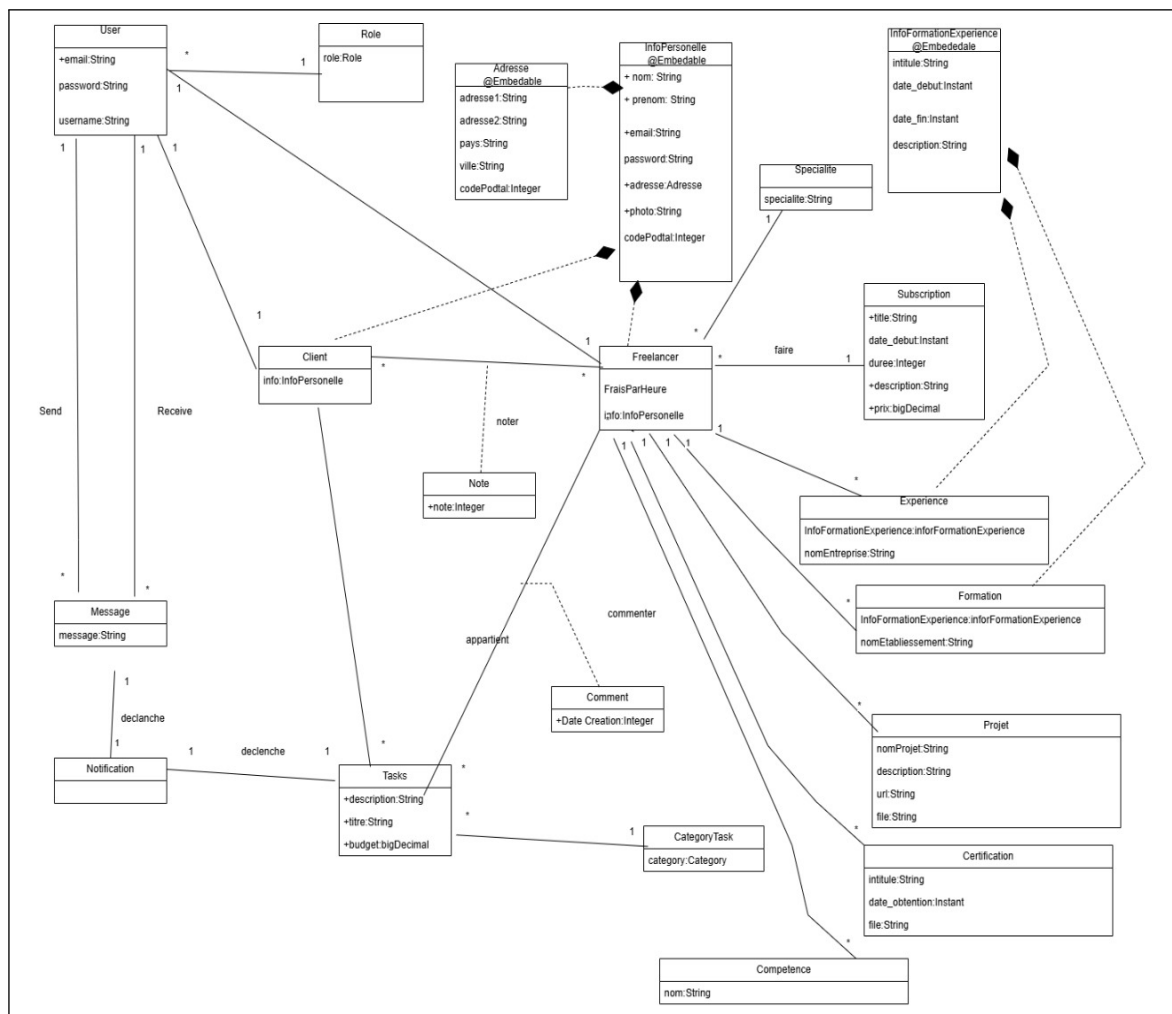


FIGURE 3.2 – A class diagram

Production

Sommaire

4.1	Introduction	25
4.2	DevOps Integration in the Project	30
4.3	1. Unit Testing Integration	30
4.4	2. Code Quality Analysis with SonarQube	30
4.5	Conclusion	30

4.1 Introduction

In this chapter, we're going to present the implementation part of our project, using using screenshots of the site's interfaces.

- P.S. Any functionality that is unclear in the visual representation will have a textual description.

1) Login/Sign in page

a) Login / Sign in

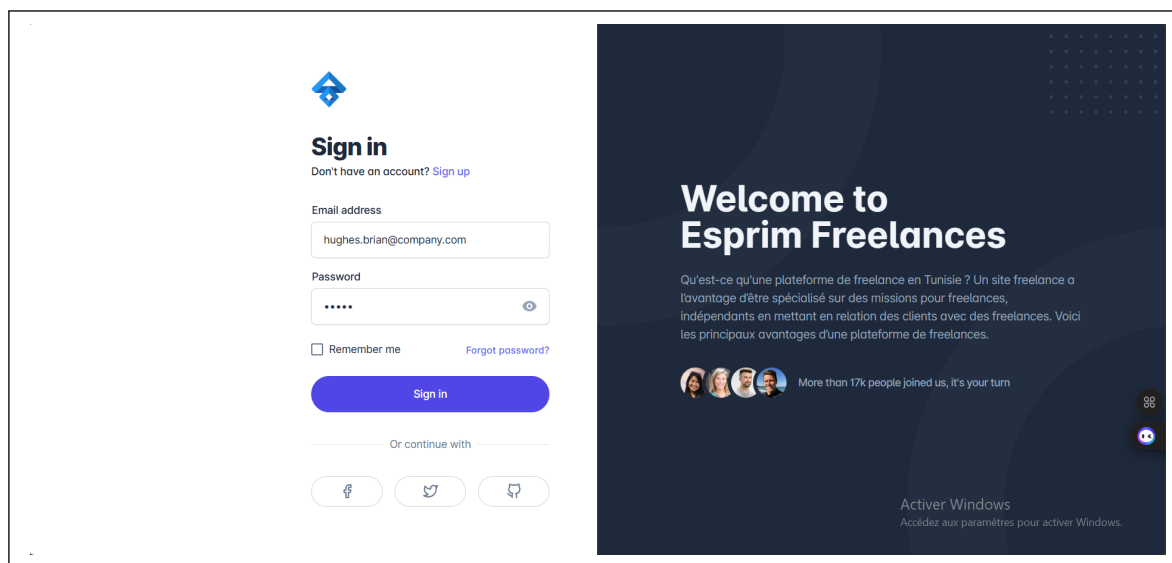


FIGURE 4.1 – Login / Sign in ui

- On this page, the user has the option to sign in using the forum on the right, authenticate using the forum on the left, or reset his password.

b) Forget password

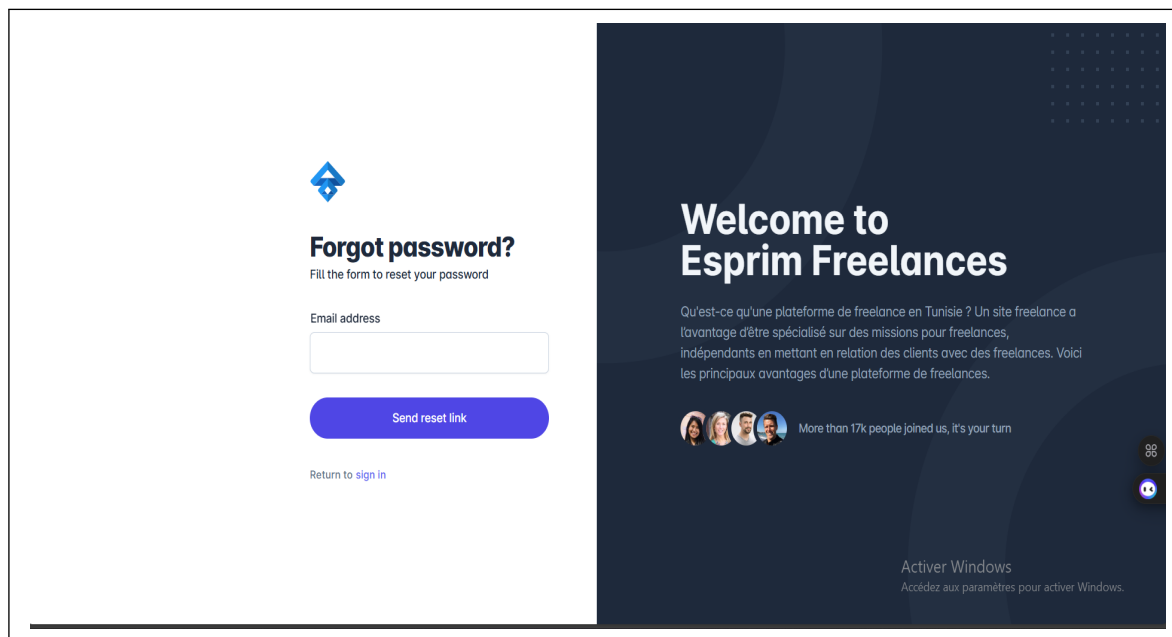


FIGURE 4.2 – Forget password ui

- The user can reset his password using this forum.

2) Mail page

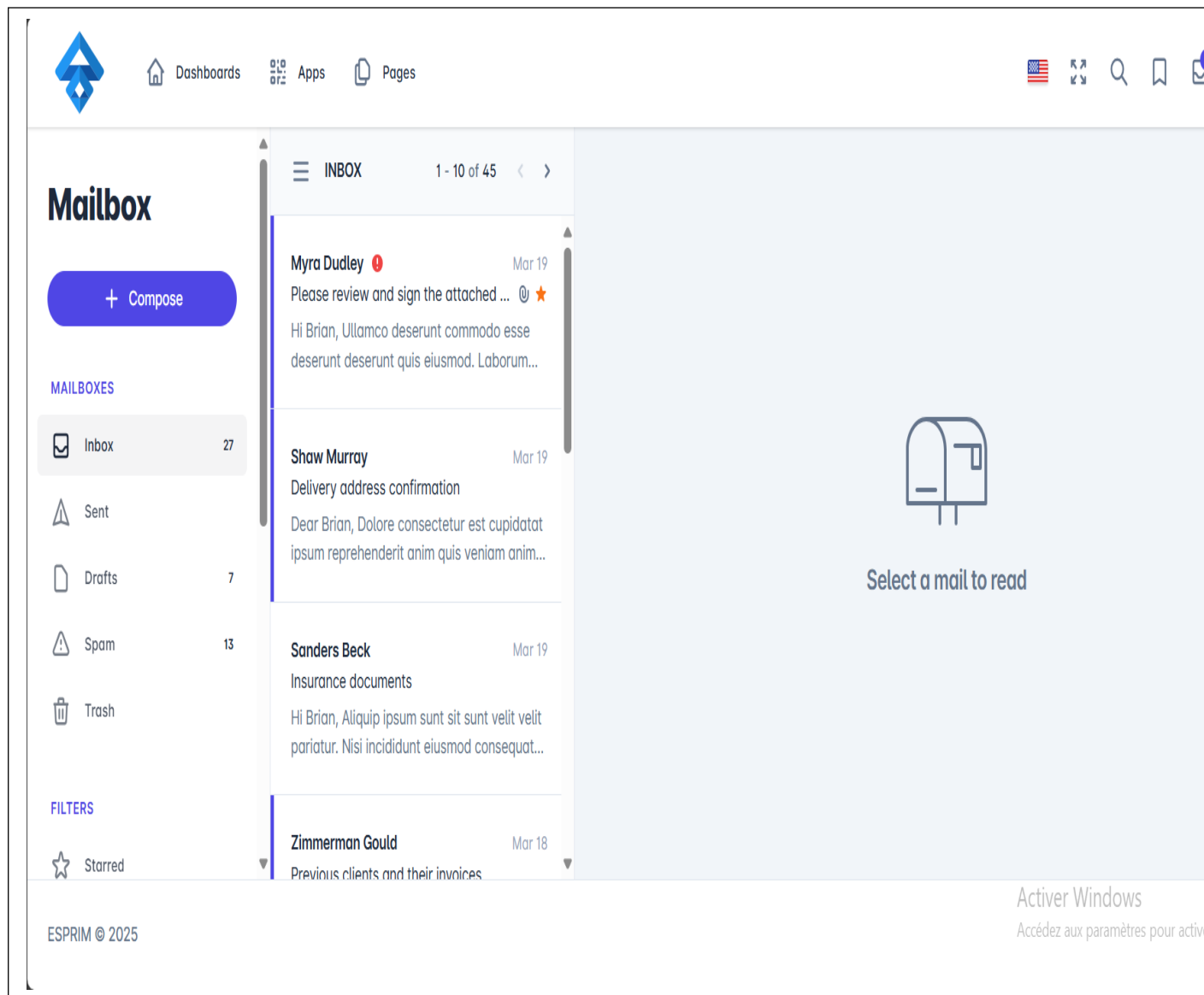


FIGURE 4.3 – Mail page ui

a) Header

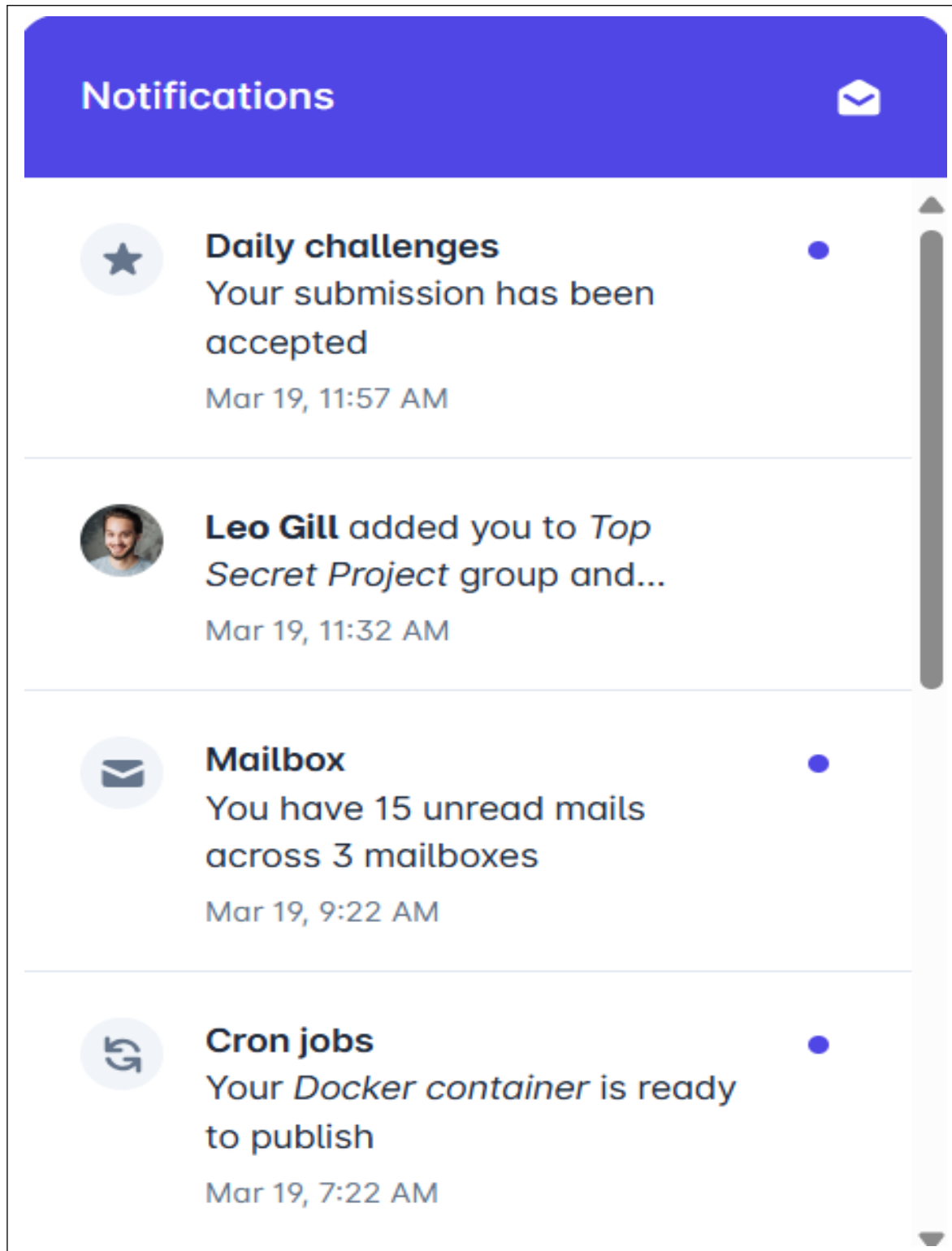


FIGURE 4.4 – Notification ui

b) Task section

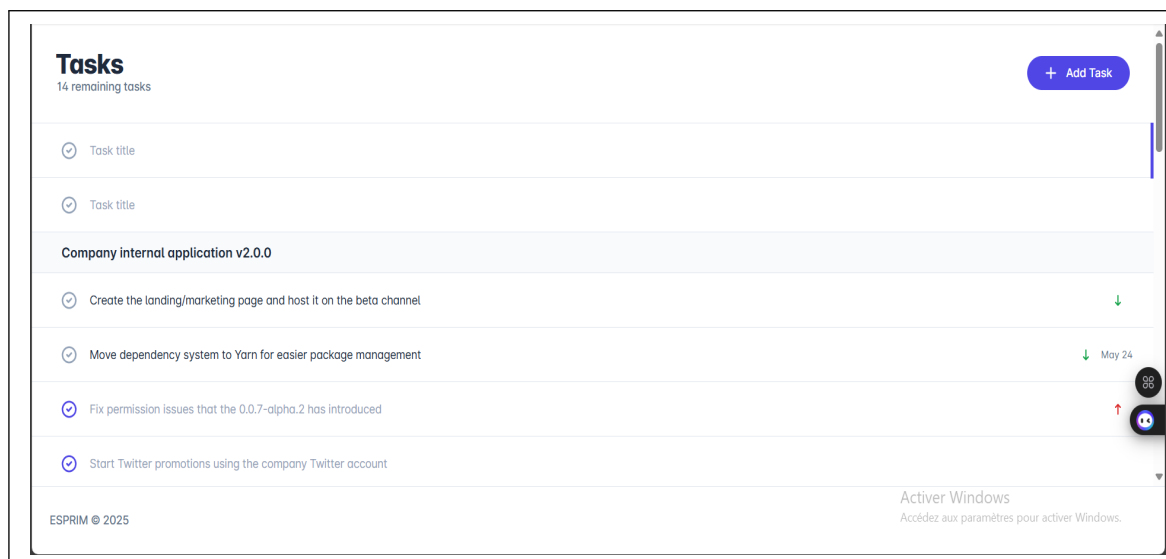


FIGURE 4.5 – Task section ui

4.2 DevOps Integration in the Project

The integration of DevOps practices significantly enhanced the quality, reliability, and deployment speed of our application. Below are the key steps implemented in our project :

4.3 1. Unit Testing Integration

To ensure code quality, unit tests were implemented using JUnit for business logic verification and Mockito to simulate external dependencies (DAO, REST services, etc.). Example : Tested Service : UserService JUnit Test : Validated the calculation logic for the duration of a broadcast. Mockito Test : Simulated the response of the UserRepository. All tests were executed successfully, and the logs confirmed good code coverage.

4.4 2. Code Quality Analysis with SonarQube

We configured SonarQube to automatically analyze our source code via Maven. The analysis helped us identify potential bugs, code duplications, and assess the level of test coverage. Key Metrics Observed : Test Coverage : 75Code Duplication : 1.2Vulnerabilities : 0

4.5 Conclusion

In this last chapter, we described the creation of our websites, which shows the different interfaces of our websites.



General Conclusion

In today's fast-paced world, the demand for well-organized camping experiences and easy access to camping equipment has increased significantly. Despite the growing popularity of outdoor activities, existing solutions often lack the integration needed to seamlessly connect campers with events and essential gear. Our project addresses this gap by providing a unified platform where users can both organize or join camping programs and purchase necessary tools within the same ecosystem.

Through this platform, we enable camping enthusiasts to easily discover and participate in events while ensuring that they have access to all the required equipment without hassle. At the same time, event organizers can efficiently manage their programs, communicate with participants, and enhance the overall camping experience. The integrated marketplace further streamlines the process by offering a specialized shopping space tailored to the needs of campers.

By implementing this project, we aim to facilitate outdoor activities, encourage community engagement, and improve the overall camping experience in Tunisia and beyond. With a well-structured system that includes user management, program organization, and e-commerce functionalities, we believe this platform will bridge the existing gaps and offer a reliable, user-friendly, and innovative solution for the camping community.